

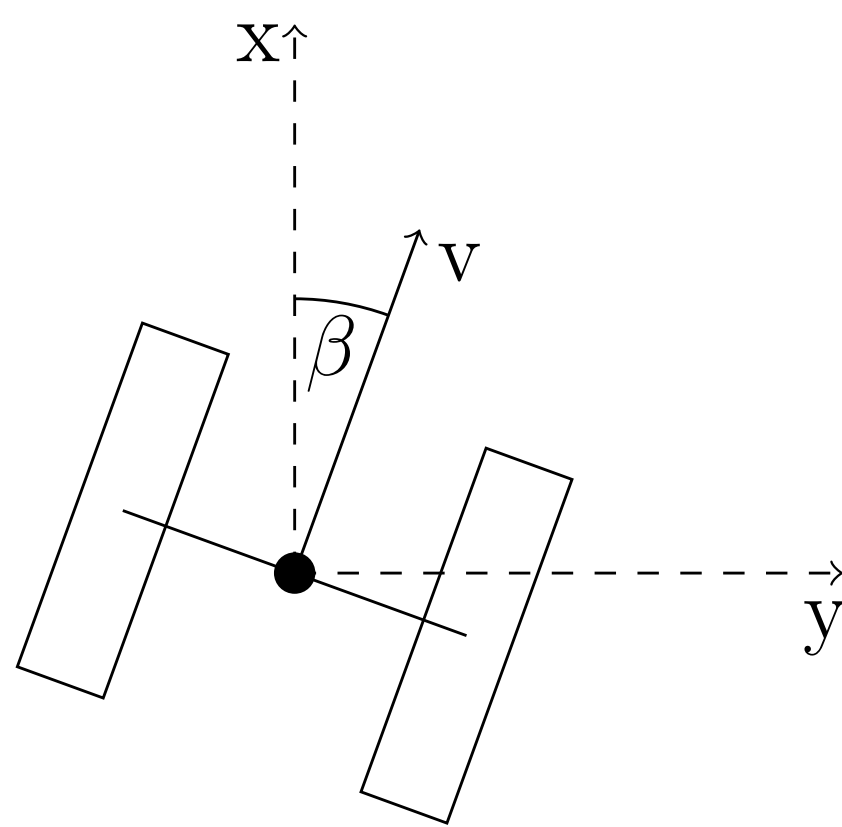
Exercise 8: Nonlinear Least Squares (to be returned on Jan 13th, 8:15)

Prof. Dr. Moritz Diehl, Katrin Baumgärtner, Jakob Harzer, Dan Wang,
Ashwin Karichannavar, Premnath Srinivasan

In this exercise you will learn how to practically solve nonlinear least squares problems, compute estimates of the covariance of your estimated parameters and make statements about the correctness of the model assumptions.

Parameter estimation for output error minimization

In this exercise we will consider the model of a differential drive robot with unicycle dynamics. The movement of the robot depends on the angular velocities of the left and the right wheel ω_L and ω_R , as well as on their radii R_L and R_R . Differing radii influence the behaviour of the robot.



The system can be described by a state space model with three internal states. The state vector $\mathbf{x} = [x, y, \beta]^T$ contains the position of the robot in the $X - Y$ plane and the deviation β from its initial orientation. The system can be controlled by the angular velocities of the wheels: $\mathbf{u} = [\omega_L, \omega_R]^T$. The output of the system is the position of the robot: $\mathbf{y} = [x, y]^T$. The model follows as

$$\dot{\mathbf{x}} = \begin{bmatrix} v \cdot \cos \beta \\ v \cdot \sin \beta \\ \frac{\omega_L R_L - \omega_R R_R}{L} \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} x \\ y \end{bmatrix} \quad (1)$$

with L being the length of the axis between the two wheels and the velocity v being

$$v = \frac{\omega_L \cdot R_L + \omega_R \cdot R_R}{2}.$$

The system state is $\mathbf{x} = [x, y, \beta]^T$ and is equal to the robot's pose. The system can be controlled by the angular velocities of the wheels: $\mathbf{u} = [\omega_L, \omega_R]^T$. The output $\mathbf{y}$ is the position of the robot and measured with a sampling time of $\Delta t = 0.01$ s.

We already provided you some functions to simulate the position of the two-wheel-robot using the state space model. For reference use chapter 6.2 'Numerical Integration Methods' from the lecture notes. Please go through all of the provided functions and try to understand what they are doing.

- `robot_ode(x,u,p)` evaluates the right-hand side of the ODE $\dot{x} = f(x, u, p)$, with parameters $p = [R_L, R_R, L]$. Use the following values: $R_L = 0.2$ m, $R_R = 0.2$ m and $L = 0.6$ m.

- `euler_step(h, x0, u, ode, p)` performs one Euler integration step for a general ODE $\dot{x} = f(x, u, p)$ starting at x_0 , with input u , parameters p and step size h .
- `sim_euler(t, x0, u, ode, p)` simulates the robot's behaviour at times $\mathbf{t}$ given a set of inputs $\mathbf{u}$, starting at $x_0 = [0 \ 0 \ 0]^\top$.

In this task, we would like to estimate the dimensions of the robot $\theta = [R_L, R_R, L]^\top$ using least squares. Assuming that the robot system has only output errors, and that these errors are Gaussian with zero mean and variances $\sigma_x^2 = 1.6 \cdot 10^{-3} \text{ m}^2$ and $\sigma_y^2 = 4 \cdot 10^{-4} \text{ m}^2$, then the Maximum Likelihood Estimation problem to estimate θ is:

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^3} \sum_{k=0}^N \|\mathbf{y}_k - M_k(\mathbf{U}, \mathbf{x}_0, \theta)\|_{\Sigma_y^{-1}}^2,$$

where $\mathbf{y}_k = (x, y)^\top \in \mathbb{R}^2$ with x and y being the coordinates of the robot and N is the number of measurements; Σ_y is the weighing matrix containing the variances on the x and y measurements,

$$\Sigma_y = \begin{bmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_y^2 \end{bmatrix};$$

$M_k(\mathbf{U}, \mathbf{x}_0, \theta)$ denotes the modeled position at timestep k for given $\mathbf{U}, \mathbf{x}_0, \theta$ where $\mathbf{U} \in \mathbb{R}^{(N-1) \times 2}$ is a matrix that contains all applied control inputs $u_1, \dots, u_{N-1}$, each consisting of the angular velocity of the left and right wheel respectively (ω_L and ω_R); $\mathbf{x}_0$ contains the robot's initial pose $\mathbf{x}_0 = [x_0, y_0, \beta_0]^\top = [0, 0, 0]^\top$ which we assume to be perfectly known.

1. ON PAPER: Formulate the output model

$$M_k : \mathbb{R}^{(N-1) \times 2} \times \mathbb{R}^3 \times \mathbb{R}^3 \rightarrow \mathbb{R}^2, (\mathbf{U}, \mathbf{x}_0, \theta) \mapsto \hat{\mathbf{y}}_k$$

where you may use a function $F : \mathbb{R}^3 \times \mathbb{R}^2 \times \mathbb{R}^3 \rightarrow \mathbb{R}^3$ to denote the discretized system dynamics describing the mapping $(x_k, u_k, \theta) \mapsto x_{k+1}$. (2 point)

2. CODE: Implement a function

`residual(theta, x0, U, t, y, sigma_y)`

which computes the residual vector between the given measured locations $[\mathbf{y}_0, \mathbf{y}_1, \dots, \mathbf{y}_N]^\top$ and the modeled locations $[M_0, M_1, \dots, M_N]^\top$. Keep in mind to incorporate the measurement variances Σ_y correctly, i.e. weight the residual and to perform the right number of integration steps. Check the provided code for additional information on the parameters. (1 point)

3. CODE: Use `least_squares` in package `scipy.optimize` to solve the nonlinear least-squares problem to obtain θ^* .¹ (1 point)
4. CODE: Compute the simulated trajectory using θ^* and use the provided code to plot it versus the measurements and a 4th order polynomial fit.
ON PAPER: What do you observe? (1 point)
5. ON PAPER: Check if the assumptions made on the noise were correct by plotting a histogram for the residual in x and y (using θ^*). (1 point)
6. CODE: Approximate the covariance matrix Σ_{θ^*} of your estimate θ^* (check page 52 of the lecture notes). (2 points)

¹`least_squares` takes as input a vector function $f(\theta) = [f_1(\theta), \dots, f_N(\theta)]$, and minimizes $\frac{1}{2} \|f(\theta)\|_2^2$ with respect to θ . Thus, you have to stack the residuals obtained for different timesteps to obtain a *single* residual vector.

7. ON PAPER: Suppose you have identified the robot's kinematic model as done in this task. You are now given another series of controls $\mathbf{U} \in \mathbb{R}^{(N-1) \times 2}$ and are asked to predict the robot's end pose and to give an estimate for the covariance matrix $\Sigma_{\hat{\mathbf{y}}_N}$ of your prediction. Describe how this can be done. You may use any result or quantity introduced or computed in the previous tasks. (2 points)

$$\dot{\mathbf{x}} = \begin{bmatrix} v \cdot \cos \beta \\ v \cdot \sin \beta \\ \frac{\omega_L \cdot R_L - \omega_R \cdot R_R}{L} \end{bmatrix} ; \mathbf{y} = \begin{bmatrix} x \\ y \\ \beta \end{bmatrix}$$

$$v = \frac{\omega_L \cdot R_L + \omega_R \cdot R_R}{2} , \mathbf{x} = \begin{bmatrix} x \\ y \\ \beta \end{bmatrix} ; \mathbf{u} = \begin{bmatrix} \omega_L \\ \omega_R \end{bmatrix} ; dt = 0.01s$$

a). $\bar{\mathbf{F}}(\mathbf{x}_k, \mathbf{u}_k, \theta) \rightarrow \mathbf{x}_{k+1}$

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{b}\mathbf{u}$$

$$\mathbf{y} = (\mathbf{x} + \mathbf{d}\mathbf{u}) \rightarrow \mathbf{0}$$

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{b}\mathbf{u} \\ \mathbf{y} = \mathbf{C}\mathbf{x} \end{cases}$$

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} \frac{\omega_L \cdot R_L + \omega_R \cdot R_R}{2} \cdot \cos \beta \\ \frac{\omega_L \cdot R_L + \omega_R \cdot R_R}{2} \cdot \sin \beta \\ \frac{\omega_L \cdot R_L - \omega_R \cdot R_R}{L} \end{bmatrix}$$

$$\mathbf{y} = \begin{bmatrix} x \\ y \end{bmatrix} \rightarrow \mathbf{y} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ \beta \end{bmatrix}$$

→ Discretization

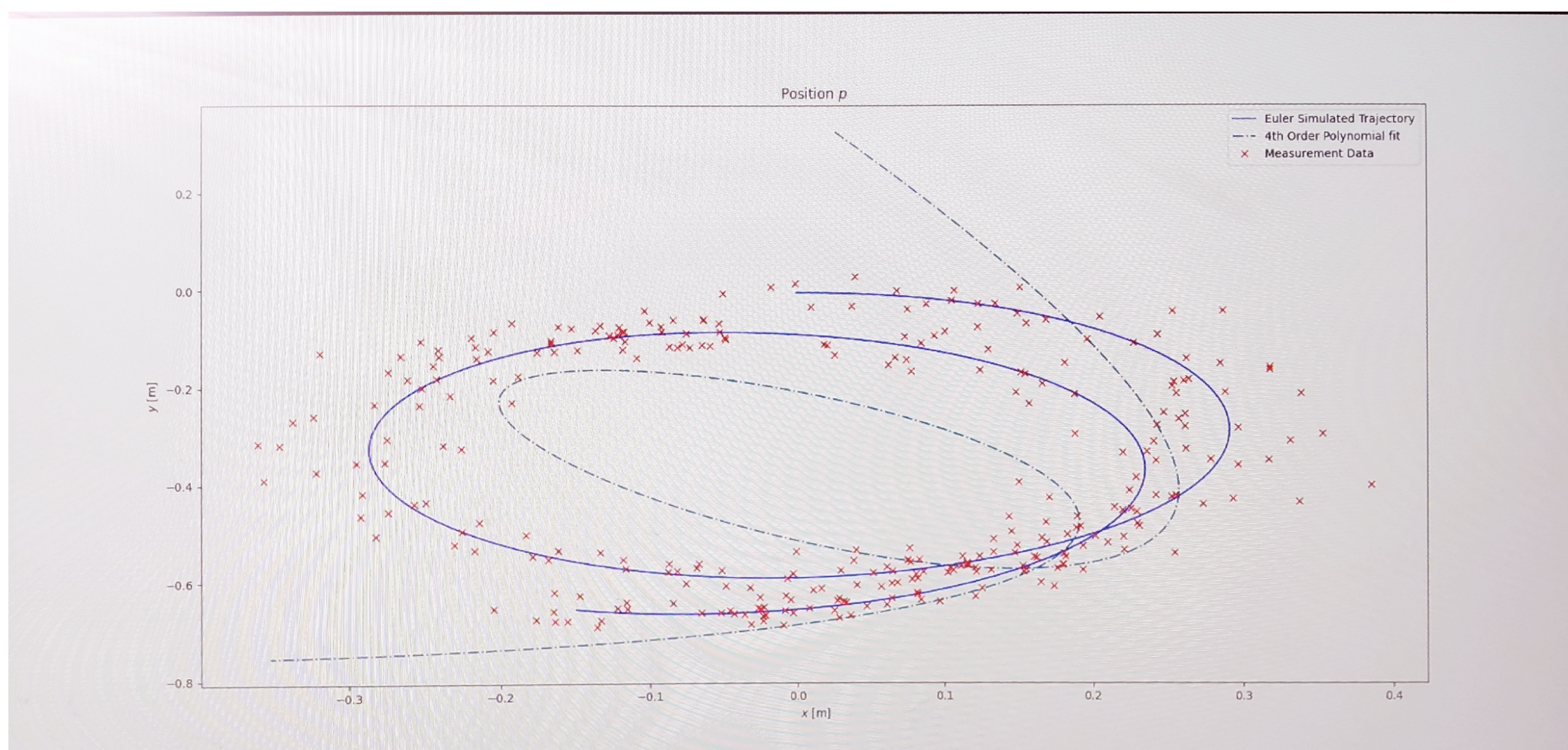
$$\mathbf{y} = \begin{bmatrix} \Delta t & 0 & 0 \\ 0 & \Delta t & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_k \\ y_k \\ \beta_k \end{bmatrix}$$

$$\dot{x} = \frac{x_{k+1} - x_k}{\Delta t}$$

$$\rightarrow \frac{x_{k+1} - x_k}{\Delta t} = \begin{bmatrix} \frac{w_L \cdot R_L + w_R \cdot R_R}{2} \cdot \cos \beta \\ \frac{w_L \cdot R_L + w_R \cdot R_R}{2} \cdot \sin \beta \\ \frac{w_L \cdot R_L - w_R \cdot R_R}{2} \end{bmatrix} \Delta t + x_k$$

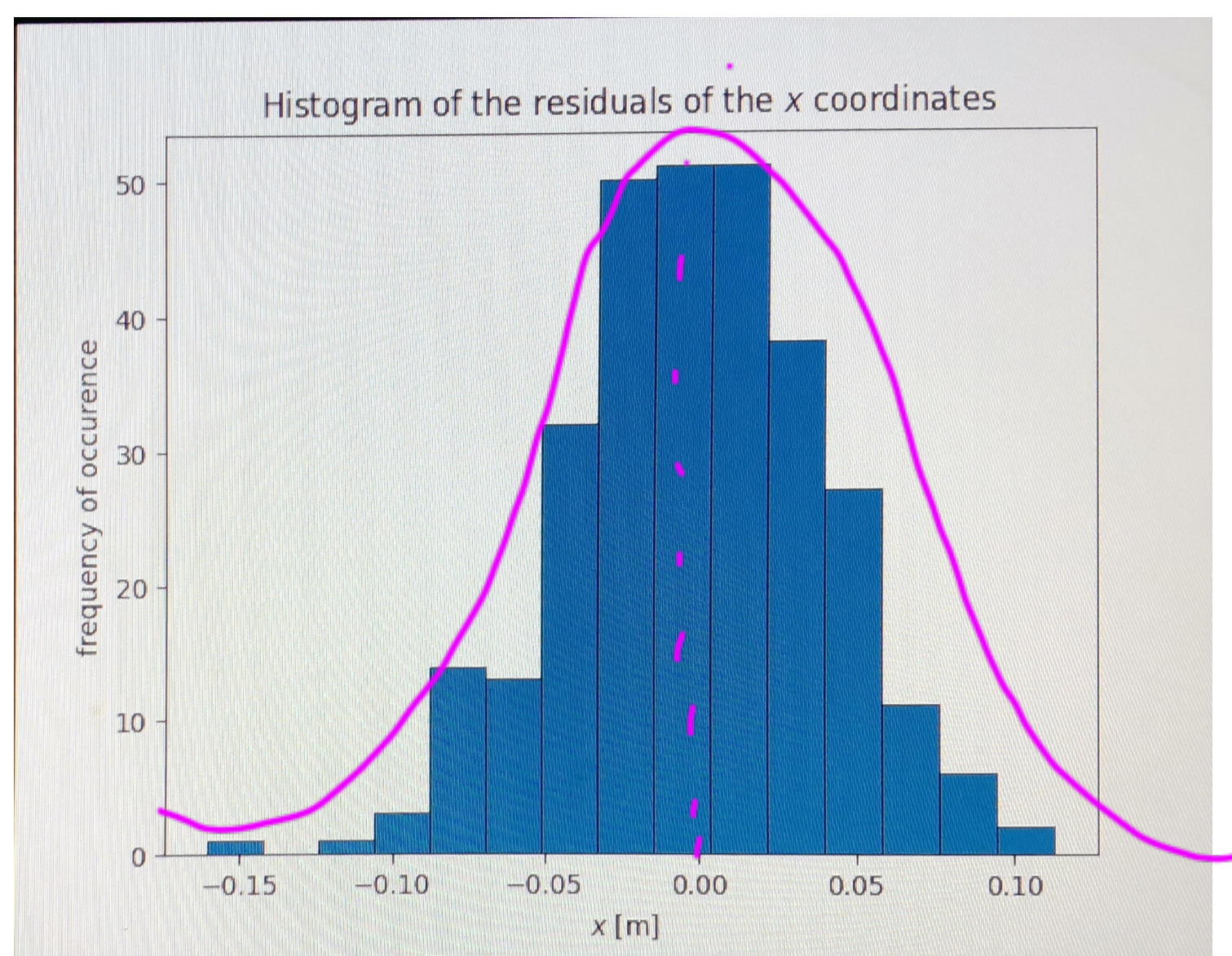
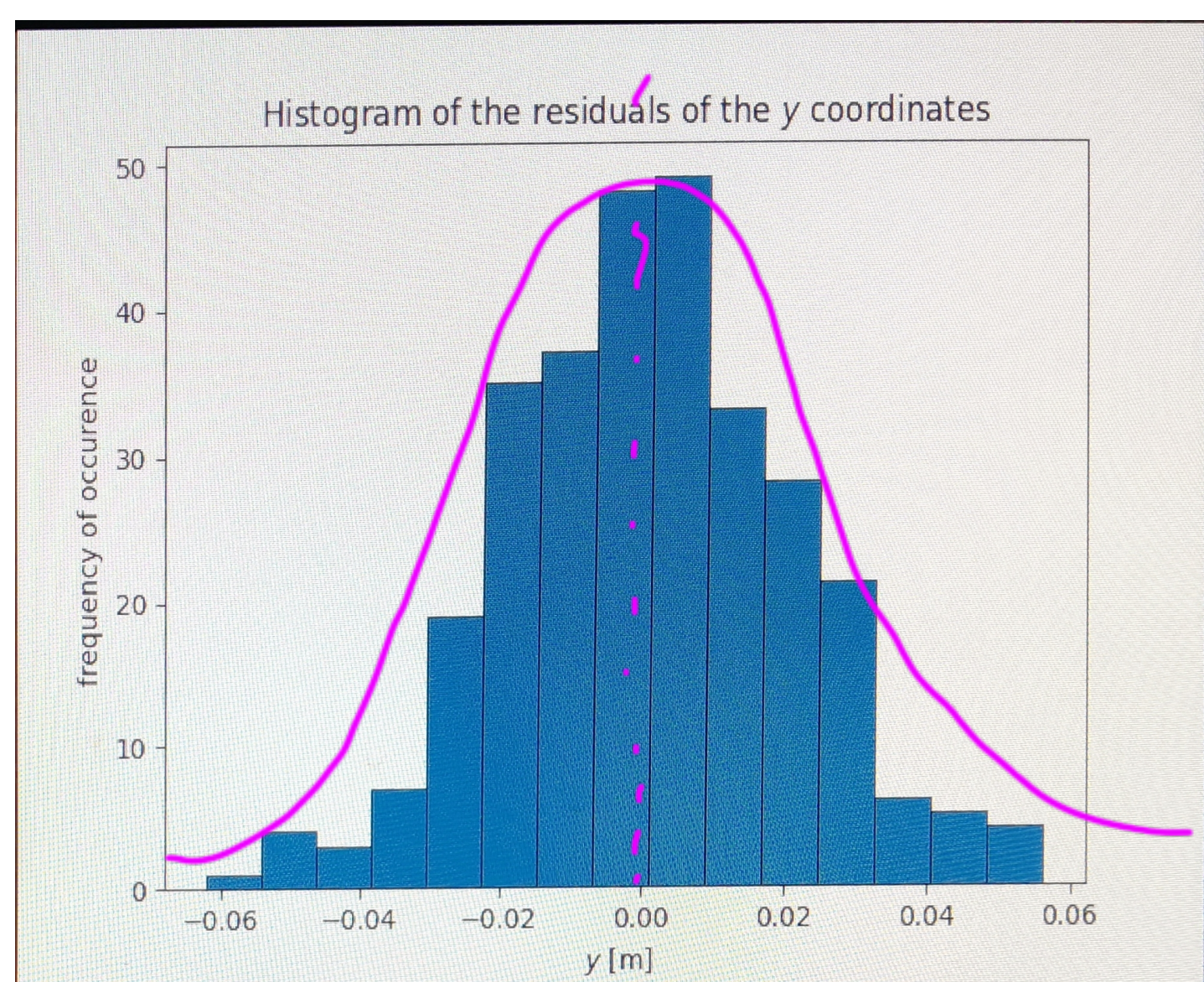
$x_{k+1} =$

4). Observation



- the 4th order polynomial has the same shape with the robot's movements. However, the polynomial fit poorly tracks the measured data of the robot.
- On the other hand, the simulated Euler fits extremely well with the measurements of the car.

5)



→ The Assumptions are correct → we can see that the

$$v_x \sim \mathcal{N}(0, \sigma_x^2)$$

$$v_y \sim \mathcal{N}(0, \sigma_y^2)$$

2. 1. Prediction of Robot's movement

known: θ^* (computed), controls U

$\Rightarrow \text{sim_euler} = (t, x_0, U; \theta^*)$

$\rightarrow$ propagate the robot in time with given U , x_0 and estimated θ^* .

2. check the sensitivity of prediction (J_N).

- sensitivity matrix J_N .

- This J_N is different from the training matrix J of non linear least square

- Introduce a small amount of errors to the estimated parameters and simulate the system forward in time steps.

- compare the differences between the simulations with and without the errors then divide to the introduced errors $\rightarrow$ resulting J_N matrix.

3. propagate uncertainty: $\Sigma_{\hat{y}_N} = J_N \cdot \Sigma_{\theta^*} \cdot J_N^T$